

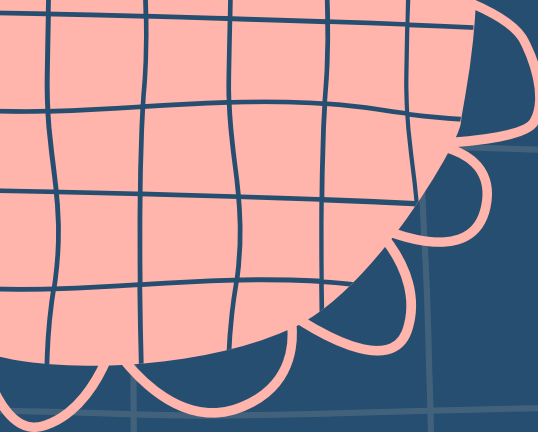


SYSCALLS & INTRO TO XV6

OSN Tutorial -1

presented by
Mohit and Nandini





WHAT IS A SYSCALL?



A system call (syscall) is a request made by a user program to the operating system to perform a task that requires special privileges.

- 
- Reading or writing a file
 - Creating a new process
 - Allocating memory
 - Communicating with hardware

Gateway between applications and the operating system.

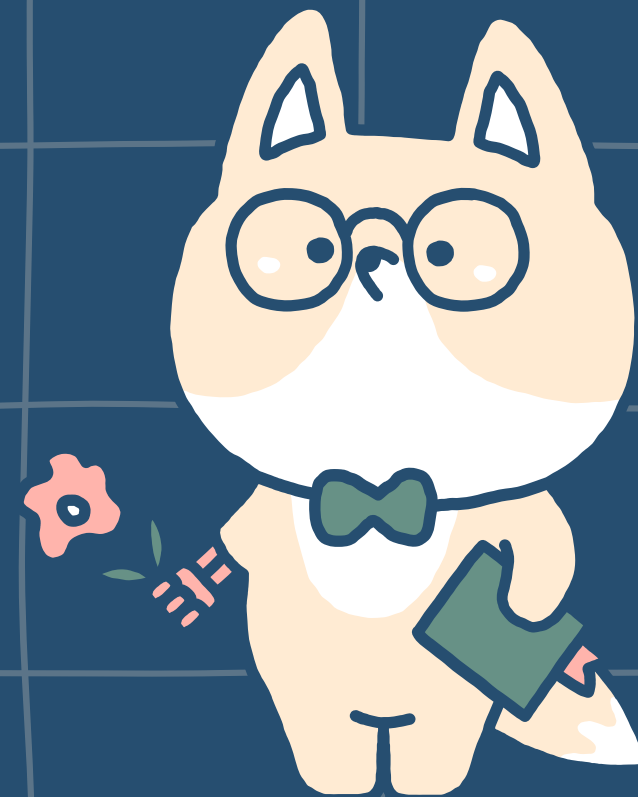


WHY SYSCALL?

User programs run with **limited privileges** and cannot directly access hardware or perform sensitive operations.

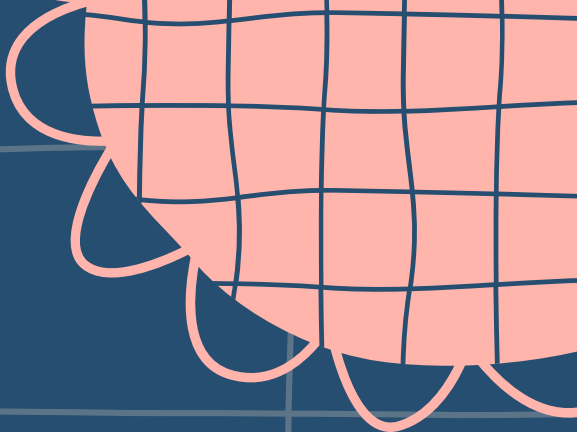
Syscalls provide a safe and controlled way to request these services from the OS

- Programs could crash the system.
- Any program could access another program's memory.
- Hardware resources would not be protected.





TWO MODES OF EXEC



USER MODE

- Applications
- Restricted Privileges
- No hardware access
- Cannot modify page tables

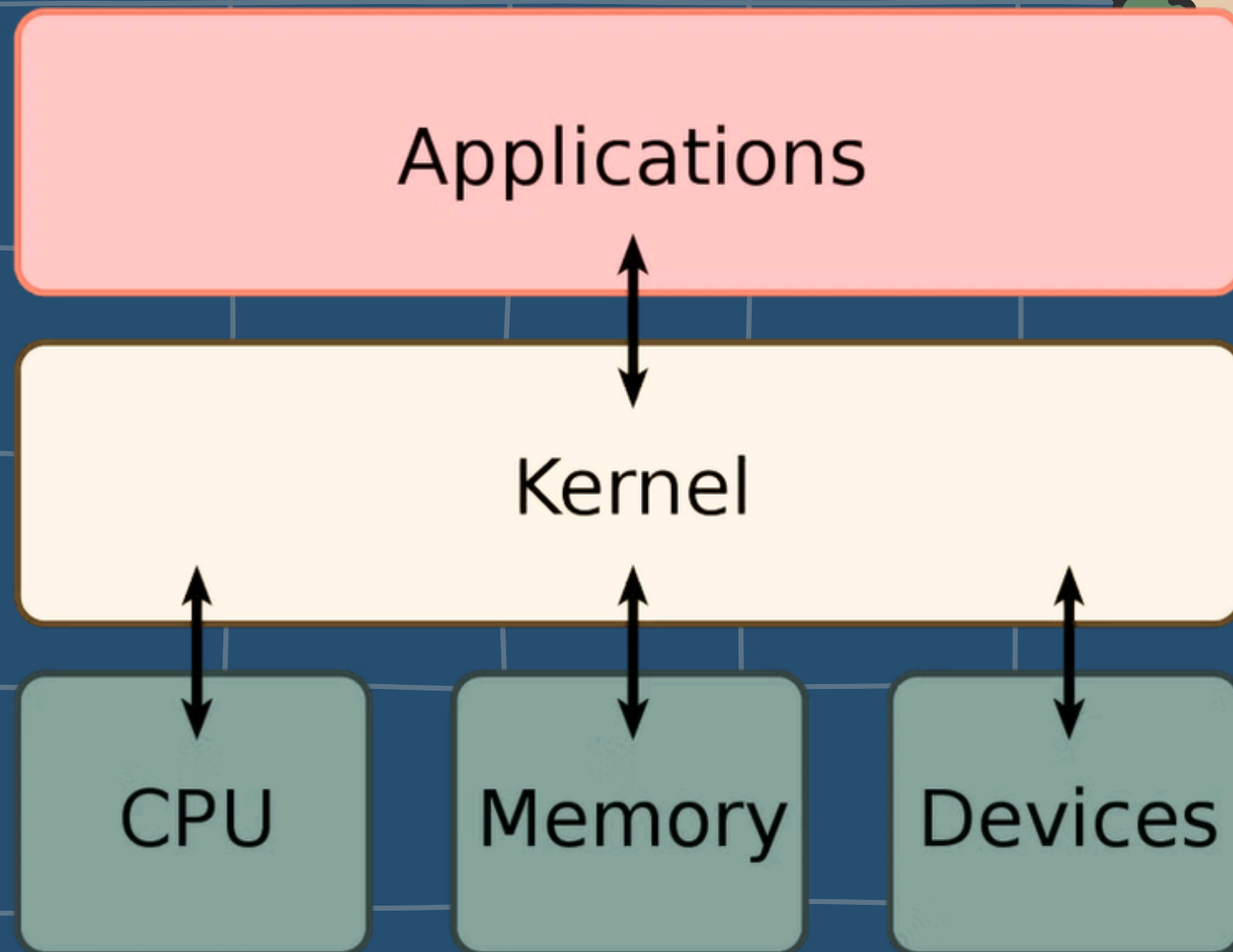
KERNEL MODE

- Operating System
- Full Privileges
- Can access hardware
- Can modify memory maps

A system call temporarily switches execution from user mode to kernel mode.



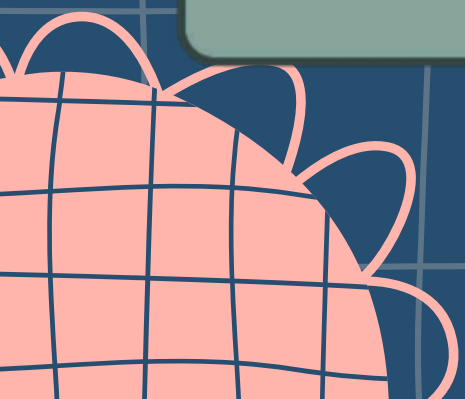
HOW DO THEY TALK?



When a program needs an OS service:

1. The program calls a system call (e.g., `read()` or `fork()`)
2. The CPU switches from User Mode to Kernel Mode.
3. The kernel performs the requested operation.
4. The result is returned to the program.
5. The CPU switches back to User Mode.

**When you try to run Popcorn.exe
and find out don't have kernel permission**



CPU ACTION

How does the CPU know when to switch from you to the kernel?

INTERRUPT

Signal sent by hardware.
(Keyboard key, Timer)

EXCEPTION

Instruction currently being executed.
(Divide by Zero)

TRAP

The program intentionally asks the CPU to enter the kernel.

What does the CPU do when a trap occurs? → It saves the program's state, switches to kernel mode, and transfers control to the trap handler.

syscalls/ecalls
(read, fork etc)

TRAP HANDLER

What exactly happens after you call write()?

User Program → Syscall → CPU saves state → Kernel Mode → Trap Handler → Execute syscall → Restore state → Back to User Program

```
if(r_scause() == 8){  
    p->trapframe->epc += 4;  
    intr_on();  
    syscall();  
} else if((which_dev = devintr()) != 0){  
    // device interrupt  
} else {  
    // unexpected exception  
}
```

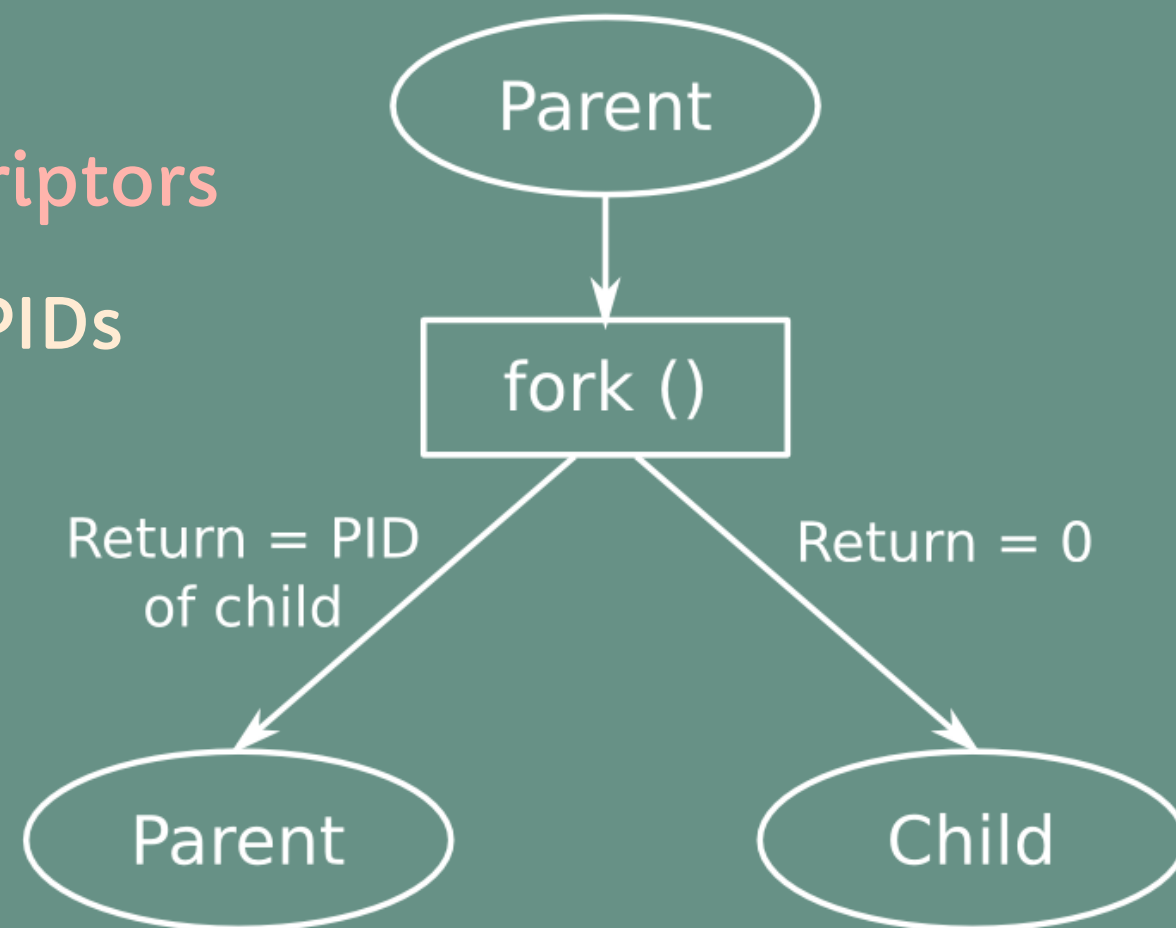
The trap handler first asks:
Is this a system call?
If not, is it a hardware interrupt?
Otherwise, it is treated as an unexpected exception.
It then forwards execution to the appropriate handler.

PROCESS

A process is a running instance of a program.

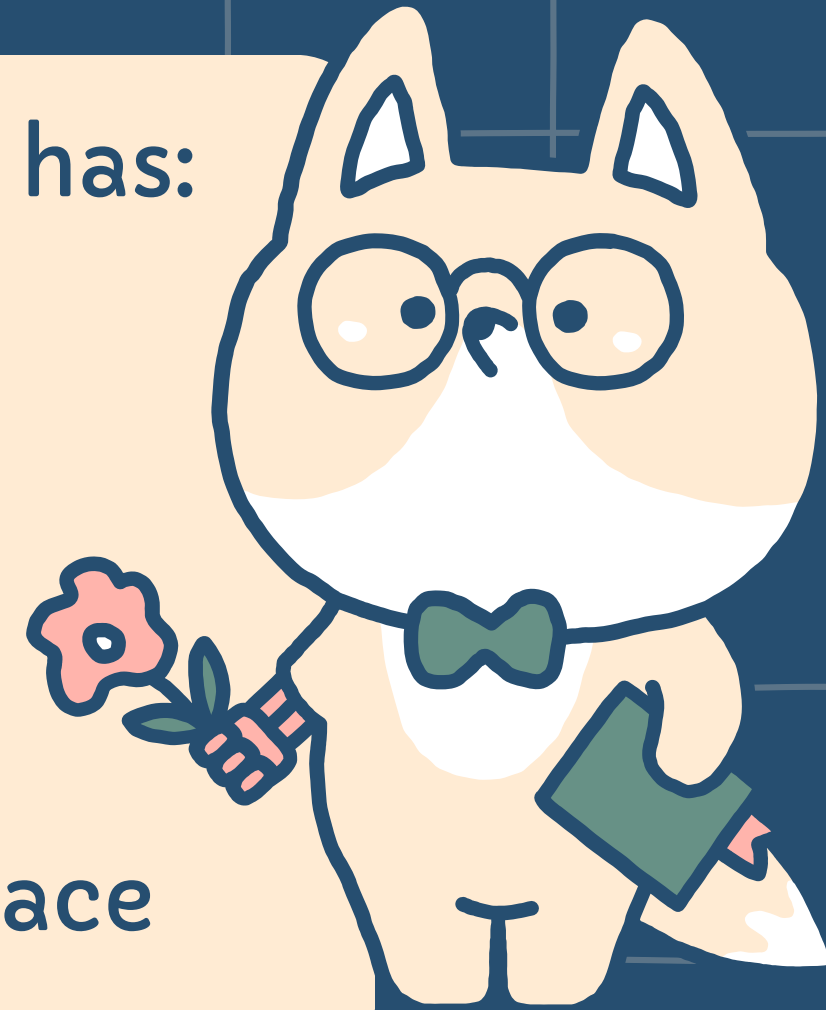
Parent-Child Relationship

- Same program code
- Same variables
- Same open file descriptors
- They have different PIDs
- They execute independently
- Parent receives the child's PID
- Child receives 0



Every process has:

- PID
- registers
- stack
- heap
- open files
- address space



Once the child is done doing the work, control returns to shell

FILE DESCRIPTORS & PROCESS CREATION

A file descriptor (FD) is a small integer that identifies an open file or I/O stream

In Unix, everything = file.

- Files
- Terminal
- Keyboard
- Pipes
- Sockets



Every process starts with three fds:

- FD 0: stdin [standard input/keyboard]
- FD 1: stdout [standard output/terminal]
- FD 3: stderr [standard error/error message]
Eg: `read(fd, .....`

THE KERNEL HAS BEEN BREACHED

POPCORN

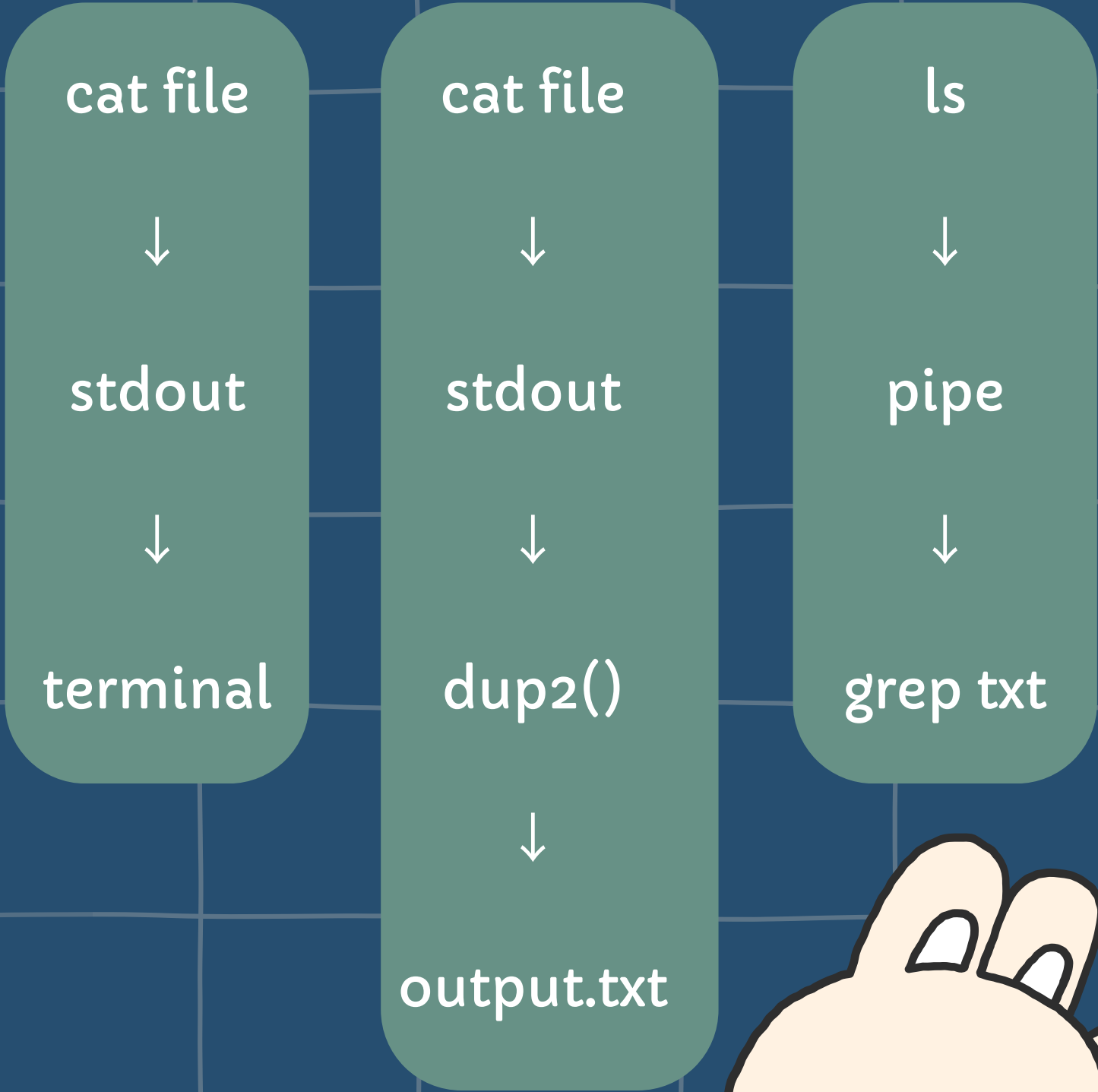
LINUX CORE DEVS



ESSENTIAL SYSCALLS

fork	Create process
exec	Run program
wait	Wait for child
exit	Terminate
getpid	Process ID
chdir	Change directory
read	Read input
write	Print output
open	Open file
close	Close file
pipe	IPC
dup2	Redirection

REDIRECTION/PIPES



INTRO TO XV6

xv6 is a small Unix-like operating system developed by MIT for teaching operating system concepts.

Kernel

- Processes
 - Memory
 - Files
 - Scheduling
 - System calls
- the core, runs in kernel mode, giving it full access to hardware.

System Call Interface

- read()
 - write()
 - fork()
 - exec()
- Whenever a program needs sm privileged, it makes a system call.

User Programmes

- sh (shell)
 - ls
 - cat
 - echo
- They run in user mode and cannot directly access hardware.



Makefile
automates
compiling xv6
'make qemu'

kernel/

- the os itself
- process management
- scheduler
- filesystem
- system calls

user/

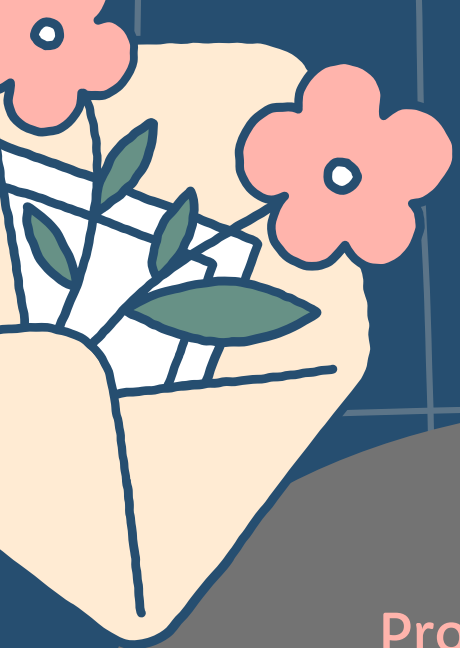
- user-space programs
- sh
- ls
- cat
- echo

mkfs/

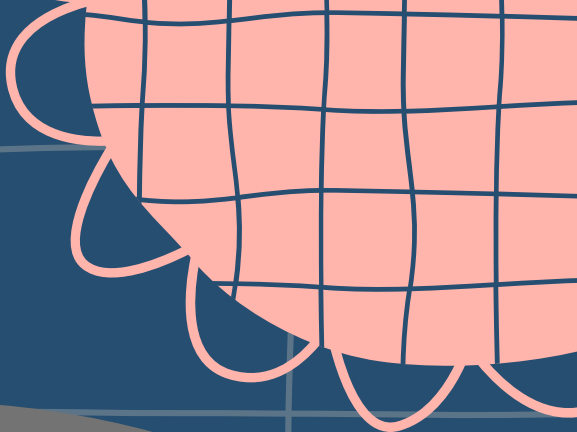
- Creates the filesystem image.
- xv6 needs a disk image containing files to start
- mkfs builds that image.

fs.img

- The actual filesystem image.
- It contains:
- directories, files
- user programs



IMPORTANT KERNEL FILES



proc.c

Process management, creating processes, scheduling, waiting most of fork() happens here

syscall.c

The syscall dispatcher, reads the syscall number, looks it up in the syscall table, calls kernel function

sysproc.c

Contains implementations of process-related system calls.

sys_fork(), sys_exit(), sys_wait(), sys_getpid()

trap.c

Whenever the CPU executes an ecall, control first comes here. This file switches from user mode to kernel mode and then invokes the syscall dispatcher.

exec.c

Implements the exec() system call.
loads a new executable, replace the current process's memory, start running the new program

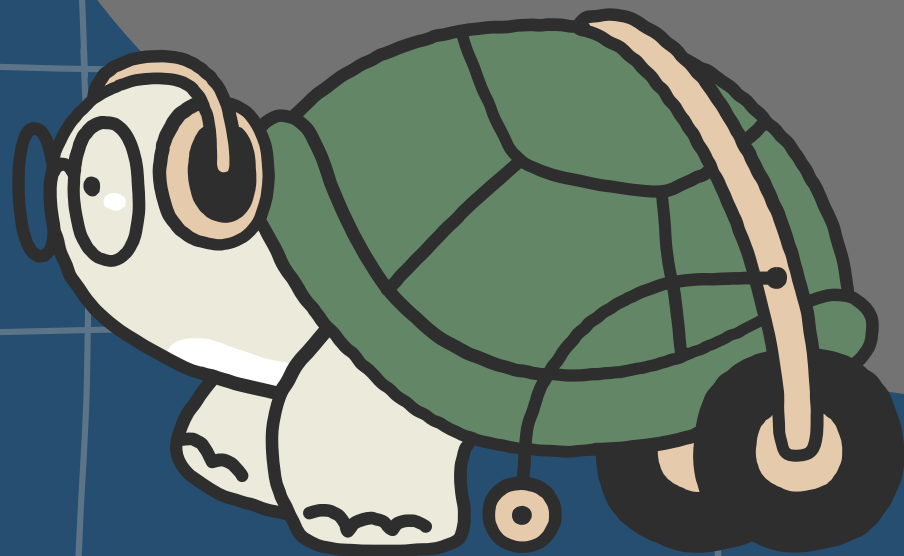
vm.c

Virtual Memory is responsible for page tables, address translation, memory allocation



TIPS FOR MP1

- Start on time
- Dont blindly use AI
- Learn something
- Have fun (●'∪'●)



THANK YOU

